Module 3

User Defined Functions

1. Write a Python program using a user-defined function word_frequency() that accepts a sentence as an argument and returns a dictionary containing the frequency of each word. Display the word that occurs most frequently.

2. Write a Python program using a user-defined function calculate_bill() that accepts the electricity consumption in units as an argument and calculates the electricity bill based on the following tariff:

Units consumed	**Rate per unit**
First 100 units	₹1.50
Next 100 units	₹2.50
Next 100 units	₹4.00
Above 300 units	₹6.00

3. Write a Python program using a user-defined function remove_duplicates() that accepts a list of elements and returns a new list containing only unique elements while maintaining their original order.

4. Write a Python program using user-defined functions to develop an Employee Payroll Management System.

The program should accept the employee ID, employee name, basic salary, and number of working days from the user.

Create the following user-defined functions:

1. calculate_hra() – Calculate HRA as 20% of the basic salary.
2. calculate_da() – Calculate DA as 15% of the basic salary.
3. calculate_pf() – Calculate Provident Fund (PF) as 12% of the basic salary.
4. calculate_salary() – Calculate the gross salary and net salary.

- Gross Salary = Basic Salary + HRA + DA
- Net Salary = Gross Salary − PF

5. display_payslip() – Display the employee details, salary components, gross salary, PF deduction, and final net salary.

Additional conditions:

- If the employee works less than 20 days, deduct 10% of the basic salary as a leave penalty.
- If the basic salary is above ₹50,000, provide a bonus of 5% of the basic salary.
- Display the final salary after applying all deductions and bonuses.

5. Write a Python program using user-defined functions to develop a Bank Account Management System.

The program should accept the account holder's name, account number, and initial balance.

Create the following functions:

- deposit() – Accept an amount and add it to the account balance.
- withdraw() – Withdraw money if sufficient balance is available.
- check_balance() – Display the current account balance.
- display_details() – Display account holder details and final balance.

Additional conditions:

- Maintain a minimum balance of ₹1,000.
- Do not allow withdrawals exceeding the available balance.
- Display appropriate messages for invalid transactions.

Graphs and Statistics

1. Student Academic Performance Analysis Using Graphs and Statistics

Consider a dataset containing student names, gender, and marks in five subjects

(Python, Java, DBMS, Mathematics and Statistics).

Write a Python program to:

1. Create a Pandas DataFrame containing student details and marks.
2. Calculate the total marks, average, median and standard deviation of each student.
3. Identify the highest-scoring and lowest-scoring students.
4. Calculate the subject-wise average marks.
5. Display a bar chart comparing the average marks of different subjects.
6. Plot a histogram showing the distribution of students' total marks.
7. Create a box plot to compare the marks distribution across subjects.
8. Display a pie chart showing the percentage of students who passed and failed (minimum 40 marks in each subject)

2. Healthcare Data Analysis Using Statistical Graphs

Consider a healthcare dataset containing patient age, gender, height, weight, blood pressure and cholesterol levels.

Write a Python program to:

1. Load the healthcare dataset into a Pandas DataFrame.
2. Identify and handle missing values.
3. Calculate the mean, median, mode and standard deviation of age, weight and cholesterol levels.
4. Calculate the BMI of each patient using height and weight.
5. Classify patients into different BMI categories.
6. Plot a histogram showing the distribution of patient ages.
7. Create a scatter plot to examine the relationship between age and cholesterol levels.
8. Generate a box plot to identify outliers in cholesterol levels.
9. Create a correlation heatmap to identify relationships among numerical

attributes.

10. Display a bar chart showing the number of patients in each BMI category.

3. Temperature Analysis

Consider the daily temperature records of a city for 30 days.

Write a Python program to:

1. Store the temperature values in a NumPy array.
2. Calculate the mean, median, maximum and minimum temperature.
3. Calculate the variance and standard deviation.
4. Display a histogram showing the temperature distribution.
5. Plot a line graph showing daily temperature variations.

4. Write a Pyhon program to:

1. Create two 3 ×3 matrices using NumPy.
2. Perform matrix addition and subtraction.
3. Perform element-wise multiplication.
4. Perform matrix multiplication using np.dot().
5. Find the transpose of both matrices.

5. Write a Python program to:

1. Generate 20 random integers between 1 and 100 using NumPy.
2. Calculate the mean, median and mode.
3. Find the variance and standard deviation.
4. Identify values greater than the mean.
5. Sort and display the generated array.

6. Write a Python program to:

1. Create a NumPy array containing 20 random integers.
2. Extract all even and odd numbers.
3. Extract numbers greater than 50.
4. Replace all negative values in an array with zero.
5. Use np.where() to replace values greater than 75 with 100.

1. Student Academic Performance Dataset Cleaning and Analysis

Consider a student dataset containing the following attributes:

Student_ID, Student_Name, Gender, Department, Attendance, Internal_Marks, External_Marks, Total_Marks

Write a Python program using Pandas to perform the following operations:

1. Load the student dataset from a CSV file.
2. Display the first five and last five rows of the dataset.
3. Display the number of rows and columns using shape.
4. Identify and count missing values in each column.
5. Remove rows where Student_ID or Student_Name is missing.
6. Replace missing attendance values with the median attendance.
7. Replace missing internal and external marks with their respective column means.
8. Remove duplicate student records.
9. Select and display only Student_Name, Department and Total_Marks columns.
10. Extract students who belong to the Computer Science department.
11. Add a new column named Result based on the total marks.
12. Delete unnecessary columns from the dataset.
13. Sort students based on Total_Marks in descending order.
14. Calculate department-wise average marks.
15. Save the cleaned dataset into a new CSV file.